Hierarchical N-Body Simulation of Galactic Dynamics in WebGPU

Zaid Alsaheb IE University, School of Science & Technology

Supervised by Professor Raul Perez Pelaez

Abstract

GPU-accelerated N-body solvers are well established on native platforms such as CUDA and Metal, but no prior work has implemented, optimised, and systematically benchmarked a hierarchical Barnes–Hut solver in WebGPU, which is the only current GPU API that combines compute shaders with browser deployability. We evaluate WebGPU as a compute platform for scientific simulation by implementing and benchmarking a complete Barnes–Hut gravitational N-body solver. The solver constructs and traverses a Linear Bounding Volume Hierarchy (LBVH) entirely on the GPU each timestep using the parallel method of Karras, combined with a symplectic leapfrog integrator. A single C++/WGSL codebase targets both native desktop execution and browser deployment via WebAssembly. We characterise performance across four WebGPU implementations (wgpu-native, Dawn, Chrome, and Safari) on an Apple M2 and compare against a native Metal Barnes–Hut baseline to isolate the cost of the WebGPU abstraction layer. At $N = 1000$ the abstraction layer imposes a 2.0 $\times$ overhead relative to native Metal; at $N \geq 5000$ the WebGPU solver outperforms the baseline, reaching 2.9$\times$ faster at $N = 100000$. Force evaluation accounts for 94–99% of total step time, with LBVH construction staying below 0.35 ms at all tested particle counts. Browser execution via Emscripten introduces a scaling overhead that narrows from 2.0$\times$ to 1.4$\times$ as GPU compute dominates, preserving the qualitative scaling behaviour seen in native execution. The 32-bit floating-point precision of current WebGPU implementations, together with the tree force approximation, constrains numerical fidelity. The results show that WebGPU's compute shader model can support a hierarchical N-body solver with parallel tree construction, and that the abstraction overhead is low enough for interactive galactic dynamics simulation without specialised hardware or vendor-locked APIs. These findings establish a performance baseline for GPU-accelerated scientific computing in WebGPU and demonstrate that browser-based deployment of computationally demanding simulations is now practical.

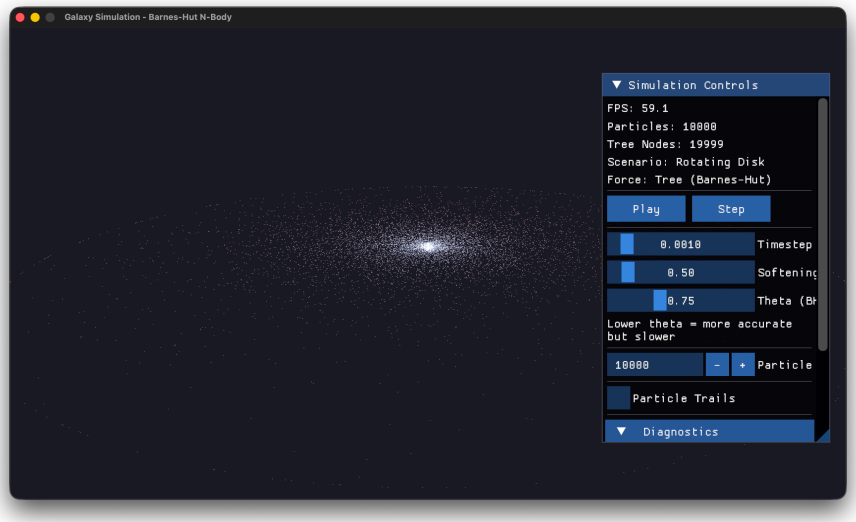

Figure 1: The interactive simulator running a rotating exponential disk.

How It Works

The solver runs a leapfrog integration step, then rebuilds the spatial tree from scratch on the GPU. To build the tree, every particle's 3D position is hashed into a Morton code and the particles are radix-sorted so that spatially close ones end up next to each other in memory. A binary bounding volume hierarchy (LBVH) is then constructed from the sorted order using Karras's parallel algorithm. Once the tree is ready, each particle walks it top-down, treating distant groups of particles as a single combined mass and only resolving nearby ones individually. This is the Barnes-Hut approximation, and it brings the per-step cost from $O(N^2)$ down to roughly $O(N \log N)$. The entire pipeline, six tree-build passes plus the force traversal, stays on the GPU with no CPU involvement.

Key Results

Almost all of the GPU's time goes into traversing the tree to compute forces, **94–99% of every timestep** depending on particle count. Building the tree itself is cheap: the six-pass LBVH pipeline finishes in under a third of a millisecond even at 100K particles. The traversal shader is where any remaining performance gains would come from.

WebGPU vs Native Metal

The most surprising result is that **our WebGPU solver beats a native Metal implementation** once the workload is large enough. We tested against UniSim, an open-source Metal Barnes–Hut code running on the same Apple M2 with the same GPU driver. At small particle counts WebGPU pays a dispatch-overhead tax, but by $N = 5000$ the fully GPU-resident tree pipeline pulls ahead, and the gap keeps widening from there.

N	Metal (ms)	WebGPU (ms)	Ratio
1,000	2.94	5.86	2.0× slower
5,000	10.09	7.35	**1.4× faster**
10,000	21.35	11.00	**1.9× faster**
50,000	121.77	65.46	**1.9× faster**
100,000	516.78	180.11	**2.9× faster**

Table 1: WebGPU (wgpu-native) vs native Metal (UniSim) on Apple M2.

Implementation choice matters

Not all WebGPU backends are equal. We ran the same solver on four implementations, all using the same Metal driver on the same machine, and saw **large performance differences**. Which backend is fastest depends on the workload size: Dawn wins at small dispatches, wgpu-native wins at scale. In the browser, Chrome is actually faster than native wgpu at low particle counts but scales worse, levelling off at roughly **1.4× overhead** for large simulations.

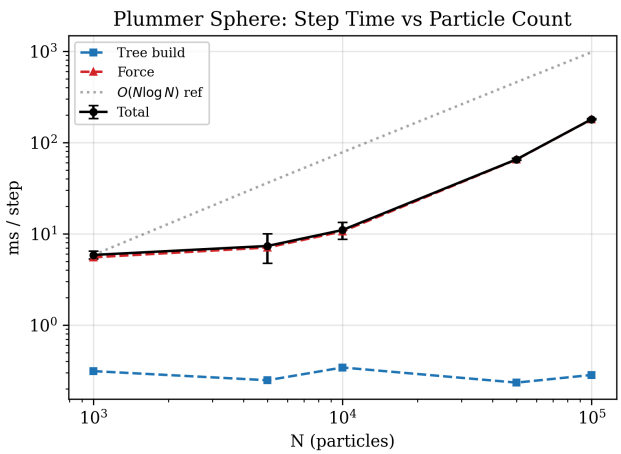

Figure 2: Runtime vs particle count (log-log).

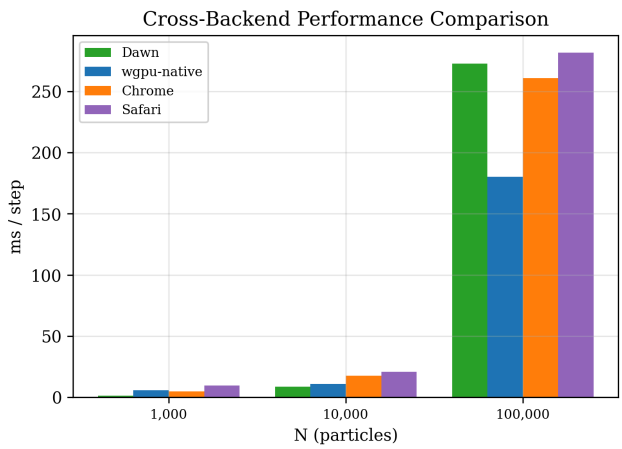

Figure 3: Four WebGPU implementations compared.